Shop as argument and not just a point, so that the new signature would look something like:

```
function insert(node, newShop, level=0)[2]
```

The application code looks so simple, but do not let it fool you: the complexity is all hidden in the `order` function, that performs all the operations concerning actually placing an order to a real shop: this likely means that you have to interact with each shop's own web applications[3] so, first of all, you have to imagine that you need a common API by which all the shops' services need to abide[4].

11.1.2 Trouble in Paradise

Now we finally have defined a method, and even a simple one, that we can use to find the closest retailer to our customer and let them know about the purchase, so that they can ship goods to users.

Notice, indeed, how simple listing 11.2 is. Simplicity, in IT as in life, is often a good thing: simple code usually means maintainable code, and flexible too. The problem is that sometimes things look great on paper, but when you deploy your application and open it to real traffic, you often find out issues you hadn't thought of or even imagined.

For example, in the real world, you have to deal with dynamic inventory for each shop and, even worse, race conditions[5].